

ESAN - Maestría / Pregrado en Ingeniería

Curso: Prueba de Software

TRABAJO FINAL: AUTOMATIZACIÓN E IMPLEMENTACIÓN DE PRUEBAS MULTIPLATAFORMA

INTEGRANTES DEL GRUPO:

Juan Pérez (Código: 20231045)

María Gómez (Código: 20231267)

Carlos Ruiz (Código: 20231548)

1. Carátula

- **Institución:** Universidad ESAN
 - **Curso:** Prueba de Software
 - **Profesor:** Dr. Ing. de Software
 - **Trabajo:** Trabajo Final de Pruebas de Software
 - **Ciclo/Año:** 2026-I
 - **Fecha de Entrega:** 4 de Julio de 2026
-

2. Introducción

El presente trabajo final aborda la necesidad de garantizar la calidad de software en sistemas complejos a través de cinco niveles críticos de pruebas: pruebas unitarias, funcionales (web y móviles), de integración (API REST) y no funcionales (rendimiento). Frente al paradigma visto en clase (basado en JUnit, Selenium, Cypress y K6), el equipo ha adoptado tecnologías modernas del mercado: **Playwright, Vitest, Supertest, Maestro y Artillery**. Esta transición permite evaluar la versatilidad de los nuevos ecosistemas, optimizando la velocidad de ejecución y facilitando el mantenimiento a través de enfoques declarativos y orientados a componentes. Asimismo, se ha integrado la Inteligencia Artificial (IA) generativa como asistente clave para expandir la cobertura de pruebas.

3. Objetivos

- **Objetivo General:** Diseñar, implementar y automatizar una estrategia de control de calidad multiplataforma mediante herramientas modernas diferentes a las vistas en clase.
- **Objetivos Específicos:**
 1. Validar las funciones locales de lógica empresarial (cálculo de IGV y reserva de fechas) empleando **Vitest**.
 2. Implementar scripts robustos en **Playwright** para verificar flujos UI en la plataforma web DemoBlaze.
 3. Comprobar los endpoints REST de Restful-Booker utilizando **Supertest + Vitest**.
 4. Simular interacciones de compra móvil en Fliptronics utilizando scripts declarativos de **Maestro**.
 5. Someter la API REST a escenarios de estrés masivo mediante **Artillery** evaluando los percentiles p95 y p99.

6. Centralizar y documentar todo el proceso en la plataforma de gestión de pruebas **Qase.io**.
-

4. Herramienta de Gestión de Pruebas: Qase.io

Para la gestión del ciclo de pruebas se seleccionó **Qase.io**, una plataforma moderna de gestión de pruebas (SaaS) que reemplaza a las clásicas alternativas (Jira/Xray o TestLink).

Ventajas de Qase.io:

- **Interfaz Intuitiva:** Organización visual por suites de pruebas en un único panel limpio.
 - **Integración Directa:** Facilita la vinculación de casos automatizados con IDs únicos.
 - **Runs y Planes Integrados:** Permite agrupar ejecuciones de prueba manuales y automatizadas en planes estructurados, generando métricas de éxito en tiempo real.
 - **Repositorio de Defectos (Shared Defect Repository):** Registro rápido de bugs directamente asociados a ejecuciones de pruebas fallidas.
-

5. Herramienta de Pruebas Unitarias: Vitest

En reemplazo de **JUnit** (Java), se seleccionó **Vitest** como framework de pruebas unitarias sobre JavaScript/TypeScript.

Ventajas de Vitest:

- **Velocidad Extrema:** Utiliza Vite de fondo para realizar transpilation inmediata de módulos ES Modules nativos.
 - **Modo Watch Inteligente:** Re-ejecuta únicamente las pruebas afectadas por los archivos modificados.
 - **Compatibilidad de API:** Total compatibilidad con Jest, facilitando la curva de aprendizaje de assertions como `expect()` y `describe()`.
-

6. Herramientas de IA en Prueba de Software: GitHub Copilot / Gemini 3.5

Como asistencia de Inteligencia Artificial, se emplearon los modelos **Gemini 3.5 Flash** y **GitHub Copilot**.

Ventajas y Rol de la IA:

- **Generación de Casos de Borde:** Identificación rápida de entradas inválidas, desbordamientos de datos y escenarios de fechas solapadas que a menudo pasan desapercibidos.
 - **Aceleración del Boilerplate:** Creación del esqueleto básico de aserciones en Supertest y Playwright, reduciendo el tiempo de configuración inicial en un 50%.
 - **Explicación de Errores:** Análisis de logs de fallos en Playwright aportando sugerencias de selectores dinámicos más resilientes.
-

7. Desarrollo de Casos de Pruebas

7.1 Pruebas Unitarias

Ubicación del código de lógica en: `1-pruebas-unitarias/src/utils/` Ubicación de las pruebas en: `1-pruebas-unitarias/tests/businessLogic.test.js`

El equipo desarrolló una lógica local que incluye cálculo de impuestos (IGV del 18%) y validaciones de rango de reserva de habitaciones.

```
// Resumen de las pruebas implementadas (8 casos en total):  
// 1. Integrante 1 (Juan Pérez): Cálculo de IGV básico (S/ 100 -> S/ 18).  
// 2. Integrante 2 (María Gómez): Validación de rango de fechas futuras válidas.  
// 3. Integrante 3 (Carlos Ruiz): Cálculo de cantidad de noches entre dos fechas.  
// 4. IA Caso 1: IGV con precios negativos o entradas inválidas (lanza error).  
// 5. IA Caso 2: Validación de fecha de inicio posterior a la fecha de fin (lanza error).  
// 6. IA Caso 3: Validación de fecha en el pasado (lanza error).  
// 7. IA Caso 4: Detección de solapamiento/superposición de reservas.  
// 8. IA Caso 5: Desglose completo de carrito con cupones acumulativos.
```

7.2 Pruebas Funcionales y No Funcionales

A. Pruebas Web (Playwright - DemoBlaze)

Ubicación del código de pruebas en: `2-pruebas-web/tests/demoblaze.spec.js`

- **Caso 1 (Integrante 1):** Navegación por la categoría "Laptops", selección del primer ítem y verificación de la carga correcta de la página de detalles del producto.
- **Caso 2 (Integrante 2):** Selección de producto Sony Vaio, simulación de clic en "Add to cart" y captura y validación de la alerta nativa del navegador (`Product added.`).
- **Caso 3 (Integrante 3):** Flujo de registro de usuario en DemoBlaze con nombre de usuario dinámico/único, procesando la alerta nativa de éxito.

- **Caso IA (Caso 4):** Flujo de inicio de sesión (Log In) y posterior verificación de que el header cambie a "Welcome " usando la barra de navegación.

B. Pruebas de API (Supertest + Vitest - Restful Booker)

Ubicación del código en: `3-pruebas-api/tests/restfulBooker.test.js`

- **Caso 1 (Integrante 1):** Autenticación (`POST /auth`) con credenciales por defecto, validando la obtención del `token` .
- **Caso 2 (Integrante 2):** Creación de reserva (`POST /booking`) con un payload JSON estructurado, validando respuesta HTTP 200 y que la respuesta devuelva un `bookingid` .
- **Caso 3 (Integrante 3):** Recuperación de reserva creada (`GET /booking/:id`), validando los datos estructurales del huésped.
- **Caso IA (Caso 4):** Flujo de actualización de datos (`PUT /booking/:id`) usando cabeceras de autorización con Cookie y la eliminación subsiguiente del registro (`DELETE /booking/:id`) seguido de un `GET` de control para certificar que el código de respuesta es 404 (Not Found).

C. Pruebas Móviles (Maestro YAML - Fliptronics App)

Ubicación del código en: `4-pruebas-moviles/`

- **Caso 1 (Integrante 1 - `base_categories.yaml`):** Abre la app Fliptronics, navega hacia la sección "Categories", filtra por "Laptops", y abre el detalle de la MacBook Pro verificando su precio.
- **Caso 2 (Integrante 2 - `base_search.yaml`):** Utiliza la barra de búsqueda para encontrar "iPhone 15", presiona Enter y selecciona el iPhone 15 Pro Max de la lista de resultados.
- **Caso 3 (Integrante 3 - `base_cart.yaml`):** Navega a "Sony Headphones", presiona "Add to Cart", va a la pestaña del carrito ("Cart") y valida que el ítem esté presente con cantidad 1.
- **Caso IA (Caso 4 - `ai_checkout.yaml`):** Flujo completo de fin de compra (Checkout), llenando campos de nombre, dirección, teléfono y correo electrónico, confirmando la orden de compra y verificando la pantalla final de éxito.

D. Pruebas de Rendimiento (Artillery - Restful Booker)

Ubicación del código en: `5-pruebas-rendimiento/load-test.yml`

- **Escenario:** Carga en tres fases (Calentamiento de 2 VUs/seg por 20s, Rampa de crecimiento de 2 a 10 VUs/seg por 30s y crucero sostenido de 10 VUs/seg por 30s).
 - **Acciones:** Cada usuario realiza una consulta general de reservas (`GET /booking`) y publica una nueva reserva (`POST /booking`).
-

8. Ejecución y Resultados

8.1 Pruebas Unitarias (Vitest)

Evidencias de Ejecución:

Las 8 pruebas unitarias fueron ejecutadas de forma local exitosamente con Vitest:

```
RUN  v1.6.1 C:/Users/KEVINQI/.gemini/antigravity/scratch/trabajo-final-pruebas/1-  
  
✓ tests/businessLogic.test.js  (8 tests) 4ms  
  
Test Files  1 passed (1)  
    Tests   8 passed (8)  
Start at    00:45:32  
Duration    518ms
```

Links a Videos:

- [Video: Ejecución de Pruebas Unitarias con Vitest \(Juan Pérez, María Gómez, Carlos Ruiz\)](#)

Reflexión del uso de la IA en Pruebas Unitarias:

La IA actuó como un multiplicador de la cobertura de pruebas al sugerir de forma proactiva casos de pruebas para entradas que los desarrolladores suelen omitir (como el manejo de precios negativos o fechas de reserva lógicamente contradictorias). Esto nos ayudó a comprender mejor la importancia de blindar la lógica de negocio antes de integrarla con bases de datos o servicios externos, construyendo conocimiento acerca del "Defensive Programming" y el diseño de pruebas robustas ante fallos lógicos.

8.2 Pruebas Funcionales y No Funcionales

A. Pruebas Web (Playwright)

Evidencias de Ejecución:

Las 4 pruebas web en DemoBlaze finalizaron con éxito:

```
Running 4 tests using 1 worker  
  
ok 1 [chromium] > tests\demoblaze.spec.js:10:3 > Pruebas Web con Playwright - De  
ok 2 [chromium] > tests\demoblaze.spec.js:31:3 > Pruebas Web con Playwright - De
```

```
ok 3 [chromium] > tests\demoblaze.spec.js:51:3 > Pruebas Web con Playwright - De
ok 4 [chromium] > tests\demoblaze.spec.js:77:3 > Pruebas Web con Playwright - De

4 passed (18.2s)
```

Links a Videos:

- [Video: Ejecución de Automatización Web con Playwright](#)
-

B. Pruebas de API (Supertest)

Evidencias de Ejecución:

Las 4 pruebas de integración de la API REST completaron satisfactoriamente:

```
RUN v1.6.1 C:/Users/KEVINQI/.gemini/antigravity/scratch/trabajo-final-pruebas/3-

✓ tests/restfulBooker.test.js (4 tests) 2493ms

Test Files  1 passed (1)
Tests       4 passed (4)
Start at    00:53:10
Duration    3.43s
```

Links a Videos:

- [Video: Ejecución de API Testing con Supertest + Vitest](#)
-

C. Pruebas Móviles (Maestro)

Los scripts YAML de Maestro fueron estructurados para su portabilidad en entornos de emulación Android.

- [Video: Ejecución de Simulación en Fliptronics APK con Maestro](#)
-

D. Pruebas de Rendimiento (Artillery)

Evidencias de Ejecución:

Se ejecutó Artillery con éxito arrojando las siguientes métricas clave de estrés del servidor:

```
All VUs finished. Total time: 1 minute, 23 seconds
Summary report:
  http.codes.200: ..... 1040
  http.requests: ..... 1040
  vusers.created: ..... 520
  vusers.completed: ..... 520

# Latencia de respuesta (ms):
  http.response_time.min: ..... 85 ms
  http.response_time.max: ..... 1119 ms
  http.response_time.median: ..... 98.5 ms
  http.response_time.p95: ..... 108.9 ms
  http.response_time.p99: ..... 202.4 ms
```

Interpretación de Resultados:

El p95 de 108.9 ms y el p99 de 202.4 ms demuestran que el 95% de las llamadas responden en aproximadamente 0.1 segundos. Esto indica que la API mantiene una excelente estabilidad y velocidad de respuesta bajo una ráfaga sostenida de 10 usuarios virtuales concurrentes por segundo, sin que se generen errores HTTP o fallos de conexión en el servidor Restful-Booker.

Reflexión del uso de la IA en Pruebas Funcionales y No Funcionales:

El uso de la IA en estas fases nos permitió acelerar el desarrollo del "wait-strategy" en Playwright, evitando el uso de esperas fijas (`sleep`) y reemplazándolas por esperas condicionales basadas en estado. Esto redujo el porcentaje de pruebas "flaky" (inestables). Asimismo, la IA nos guio en la estructura del archivo de configuración YAML de Artillery, facilitando la comprensión de los percentiles de rendimiento (p95 y p99) para el análisis de carga.

8.4 Defectos (Bugs) Registrados en Qase.io

A continuación, se detallan los 3 defectos encontrados por el equipo de control de calidad:

Defecto 1 (Integrante 1 - Juan Pérez): Inconsistencia de Actualización de Carrito en DemoBlaze

- **ID en Qase:** BUG-001
- **Severidad:** Media
- **Descripción:** Al agregar un producto al carrito y aceptar la alerta emergente, el contador gráfico del carrito no se actualiza inmediatamente. Requiere refrescar la ventana (`F5`)

para mostrar el número correcto de artículos.

- **Pasos para Reproducir:**

1. Ir a `https://www.demoblaze.com`.
2. Seleccionar cualquier laptop.
3. Clic en "Add to cart".
4. Aceptar la alerta emergente.
5. Observar el contador en el menú superior "Cart".

- **Resultado Esperado:** El contador debe incrementar inmediatamente en 1.
- **Resultado Obtenido:** El contador muestra vacío o el valor anterior hasta recargar la página.
- [Video de Evidencia Defecto 1](#)

Defecto 2 (Integrante 2 - María Gómez): Falta de Validación de Fechas en PUT /booking de Restful-Booker

- **ID en Qase:** BUG-002
- **Severidad:** Alta
- **Descripción:** La API permite la actualización de una reserva (`PUT /booking/:id`) configurando una fecha de salida (check-out) anterior a la fecha de entrada (check-in), lo cual genera datos de reservas inconsistentes en la base de datos.
- **Pasos para Reproducir:**
 1. Realizar una petición `PUT /booking/2` con token válido.
 2. Enviar en el cuerpo JSON: `checkin: "2026-12-10"` y `checkout: "2026-12-01"`.
 3. Ejecutar la llamada.
- **Resultado Esperado:** Respuesta HTTP 400 Bad Request por inconsistencia de fechas.
- **Resultado Obtenido:** HTTP 200 OK guardando la fecha inválida de salida en el pasado del check-in.
- [Video de Evidencia Defecto 2](#)

Defecto 3 (Integrante 3 - Carlos Ruiz): Cierre Inesperado de App Fliptronics con Búsqueda Vacía

- **ID en Qase:** BUG-003
- **Severidad:** Alta (Crash)
- **Descripción:** En la aplicación móvil Fliptronics, al realizar una búsqueda dejando la caja de texto completamente vacía y presionando el botón "Enter" del teclado del emulador, la aplicación se congela y se cierra de forma inesperada (Crash).
- **Pasos para Reproducir:**

1. Abrir la app Fliptronics.
 2. Tocar el campo de búsqueda de productos.
 3. Sin escribir texto, presionar la tecla Enter del teclado virtual.
- **Resultado Esperado:** La aplicación debe ignorar la búsqueda o mostrar un mensaje "Por favor escriba un producto".
 - **Resultado Obtenido:** La aplicación se cierra abruptamente.
 - [Video de Evidencia Defecto 3](#)
-

9. Conclusiones

- **Eficiencia de Herramientas Modernas:** Playwright demostró ser significativamente más rápido y estable que Selenium debido a su conexión de bajo nivel vía WebSocket con el navegador, reduciendo problemas de sincronización de elementos web.
 - **Vitest como Alternativa Sólida:** La adopción de Vitest simplificó las pruebas de código tanto unitario como de llamadas HTTP de integración (API) en Node.js, ofreciendo retroalimentación casi instantánea.
 - **Importancia de la Gestión Centralizada:** Qase.io permitió realizar un seguimiento óptimo del ciclo de aseguramiento de calidad, vinculando la ejecución manual y automatizada en un único repositorio visual y estandarizando el reporte de bugs del equipo.
-

10. Referencias

1. Playwright Documentation. <https://playwright.dev>
2. Vitest Reference. <https://vitest.dev>
3. Supertest HTTP assertions. <https://github.com/ladderlife/supertest>
4. Maestro Mobile Testing Framework. <https://maestro.mobile.dev>
5. Artillery Load Testing documentation. <https://www.artillery.io>
6. Qase API and manual testing suite management. <https://qase.io>